

Slide 1

PhishGuard - an ML phishing URL detector

- Anayo Chibuike Anyafulu - 202401813 - AI Final Assignment
- Course: Artificial Intelligence (3rd Year, Computer Science)
- Lecturer: Eng. Marciano Ombe

Speaker note: 30 seconds: greeting, one-line pitch - paste a URL, get an explained verdict, model runs in the browser.

Slide 2

Introduction: problem, motivation, objectives

- Problem: zero-day phishing outpaces blocklists; the URL is the one artefact every attack must create
- Motivation: pre-click detection that is instant, offline and privacy-preserving
- Objectives: real-data ML pipeline; lexical features only; browser deployment; honest evaluation + ethics

Speaker note: Stress 'before you click' positioning and that nothing leaves the user's machine.

Slide 3

Background in one slide

- Lexical ML detection is competitive and deployable (Sahingoz et al., 2019)
- Explainable verdicts matter for trust (Mohammad et al., 2015)
- Deployment gap: most models never ship because they need network lookups (Basit et al., 2021)
- Trees > linear on accuracy, linear > trees on transparency (Breiman, 2001; Cox, 1958)

Speaker note: Position the project as engineering of known-good theory, with citations on the slide.

Slide 4

Model architecture and design

- 18 lexical features from the URL string: lengths, counts, entropy, IP host, tld-in-subdomain, shorteners...
- RandomForest (200 trees) = accuracy benchmark | LogisticRegression = deployable product
- LR export: means, stds, weights, bias (2 KB JSON); $p = \text{sigmoid}(\text{bias} + \sum(w \cdot x))$
- Same features in Python and TypeScript, pinned by golden-master tests

Speaker note: Show the sigmoid formula; mention 6-decimal Python-browser parity as the key engineering guarantee.

Slide 5

Data and preprocessing

- 107,355 real URLs: 48,338 PhishTank phishing + 59,017 legitimate (Tranco + benign deep links)
- Fix 1: path-balanced legitimate data (else 'has a path = phishing' bias)
- Fix 2: max 10 URLs/host (else host memorisation)
- Fix 3: dedupe + stratified 80/20 split - zero leakage

Speaker note: Tell the 244-URL / F1=1.000 story here - perfect metrics were the dataset's fault, not the model's.

Slide 6

Functionality / demo

- Website: paste URL -> verdict, probability gauge, feature-occlusion explanation, full 18-feature grid
- Sample URLs for both classes; history of recent analyses
- CLI bonus: `python src/check_url.py <url> --json` (human + machine interfaces)
- Live demo: `google.com` vs `paypal.com.verify-login.ru`

Speaker note: Demo beats slides: run the two sample URLs live; point at the contribution bars while explaining.

Slide 7

Evaluation and analysis

- RF: F1 0.888, ROC-AUC 0.954 | LR: F1 0.804, ROC-AUC 0.885 (21,471 held-out URLs)
- CV F1 tracks test F1 (0.889 vs 0.888) - no split fluke, no leakage
- Confusion matrix: FN 1,091 vs FP 1,076 - errors are balanced, not collapsed
 - Top features: hostname_length 17.5%, path_length 17.4%, entropy 14.1% - plausible and inspectable

Speaker note: Honesty checks are the differentiator; recall is the metric we prioritise and we say so.

Key findings and improvements

- Deployment shapes model choice: ship LR (2 KB, exact, explainable), benchmark RF
- Data hygiene > hyperparameter tuning (capping + path balancing gave the real gains)
- Explanations (occlusion) turn a score into an argument
- Next: Public Suffix List, in-browser trees, active learning from overrides

Speaker note: One sentence: accuracy x deployability x explainability - you can maximise two, we chose deliberately.

Ethical considerations and mitigations

- FN > FP cost: recall prioritised, threshold adjustable, disclosed in-product
- Bias: multi-source data, host capping, limitations stated openly
- No auto-blocking: verdicts are advisory and overridable - human stays in the loop
- Privacy: 100% client-side inference - the URL never leaves the browser

Speaker note: End on the stance: disclose what the tool cannot see as loudly as what it can.

Conclusion

- An end-to-end AI system: real data -> two models -> honest evaluation -> deployed, explained product
- Demonstrates: problem framing, feature engineering, model selection, evaluation discipline, ethics
- All claims reproducible: one command retrains, re-evaluates and re-exports everything
- Thank you - questions welcome

Speaker note: Close with reproducibility: every number in the report regenerates from source.